Chapter 5: Implementation and Testing

5.1 Implementation Approaches

The implementation phase translates the system design specifications into a fully functional software solution. For the Payroll Management System, a modular, component-based implementation strategy was adopted using Next.js (React) for the frontend and Firebase as the backend and database layer. The system was implemented incrementally, where each module was developed, tested, and validated independently before being integrated into the complete system.

5.1.1 Implementation Strategy

The project was implemented following a layered architecture:

- Presentation Layer — Next.js React components managing the user interface, routing, and state.
- Application Layer — Business logic implemented as Next.js API routes (serverless functions).
- Data Access Layer — Firebase Firestore NoSQL database for real-time data storage and retrieval.
- Authentication Layer — Firebase Authentication providing secure, token-based user login.

Each layer was loosely coupled and interacted through clearly defined interfaces. This separation of concerns improved scalability and simplified testing. The implementation was aligned with industry coding standards, emphasizing code readability, reusability, proper documentation, and performance optimization.

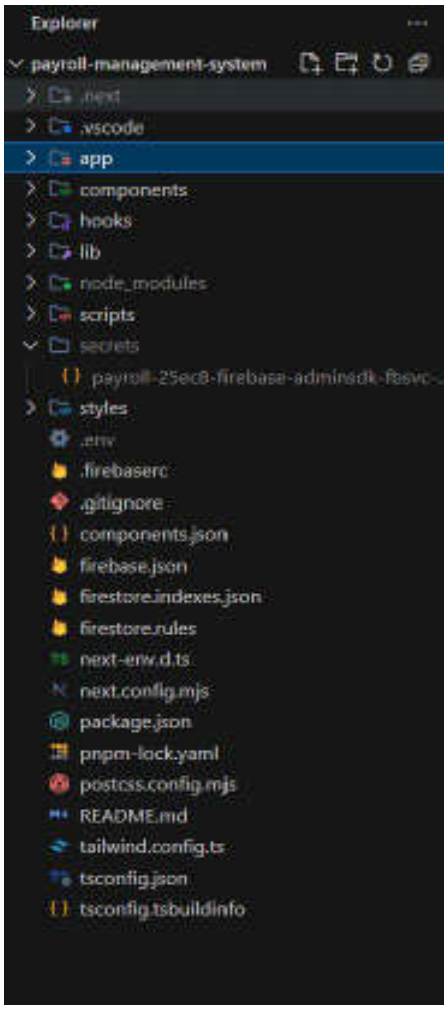

Figure 5.1 — Project folder structure in VS Code

5.1.2 Technology Stack and Standards

The following technologies were used during implementation:

- Frontend Framework: Next.js 14 (React 18) with TypeScript
- UI Styling: Tailwind CSS for responsive design
- Backend: Next.js API Routes (serverless, Node.js runtime)
- Database: Firebase Firestore (NoSQL, real-time)
- Authentication: Firebase Authentication
- Hosting/Deployment: Vercel
- Development Tools: VS Code, Git, Postman, pnpm

RESTful API principles were rigorously followed for all backend services, using standard HTTP methods (GET, POST, PUT, DELETE) for resource manipulation, stateless communication, and JSON as the data exchange format. Firebase security rules were enforced to ensure data access controls at the database level.

5.1.3 Proposed SDLC Model: Agile Methodology (Scrum)

The Agile Software Development Life Cycle was adopted for this project, focusing on continuous iteration of development and testing throughout the process. Instead of delivering the entire product at the very end, Agile breaks the project into smaller iterations (Sprints), typically lasting 1 to 4 weeks. This model allows for maximum flexibility, continuous feedback, and rapid adaptation to changing requirements.

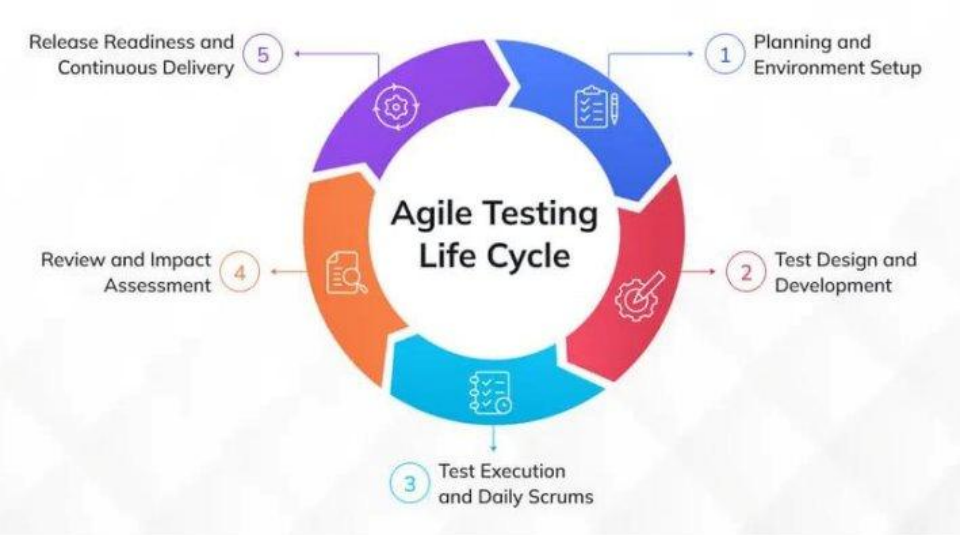

Figure 5.2 — Agile Testing Life Cycle

The following phases of the Agile cycle were adopted:

- Product Backlog Creation: All desired features were listed and prioritized — employee management, attendance tracking, payroll generation, leave requests, and reporting.
- Sprint Planning: The development team selected a priority chunk of features from the backlog for each sprint.

- The Sprint (Design, Develop, Test): Code was written and tested simultaneously within each sprint. Unit and integration testing happened continuously rather than waiting until the end of the project.
- Working Increment: At the end of each sprint, a functional, tested piece of software was ready for demonstration.
- Sprint Review and Retrospective: Completed features were reviewed against requirements and the development process was analyzed for improvements.

5.2 Coding Details and Code Efficiency

The coding phase focused on implementing critical system functionalities while maintaining optimal performance. The following section discusses the important algorithms, code segments, and techniques used in the implementation of the Payroll Management System.

5.2.1 Key Algorithm: Employee Registration and Authentication

Objective: To securely register a new employee by validating all required fields, creating a Firebase Authentication account, and storing employee details in Firestore.

Algorithm Description: The registration API route receives a POST request containing employee details. It performs input validation, creates an authenticated user in Firebase Auth, and then stores the employee profile in Firestore under a unique UID.

```
import { NextRequest, NextResponse } from 'next/server'
import { adminAuth, adminDb } from '@/lib/firebaseAdmin'

export async function POST(req: NextRequest) {
    try {
        const body = await req.json()
        const { name, email, password, position, department, salary } = body

        // Validate required fields
        if (!name || !email || !password || !position || !department || !salary) {
            return NextResponse.json(
                { error: 'All fields are required.' },
                { status: 400 }
            )
        }

        if (password.length < 6) {
            return NextResponse.json(
                { error: 'Password must be at least 6 characters.' },
                { status: 400 }
            )
        }

        // 1. Create the Firebase Auth user
        let uid: string
        try {
            const userRecord = await adminAuth.createUser({
                email,
                password,
                displayName: name,
            })
            uid = userRecord.uid
        } catch (authErr: any) {
            if (authErr.code === 'auth/email-already-exists') {
                return NextResponse.json(
                    { error: 'An account with this email already exists.' },
```

Figure 5.3 — Employee Registration API Route (Next.js)

Explanation of the code:

- Input Validation: All required fields (name, email, password, position, department, salary) are verified to be non-empty before processing.
- Password Security: Password minimum length of 6 characters is enforced at the API layer before submission to Firebase Auth.
- Firebase Auth Integration: adminAuth.createUser() creates a secure authentication entry. Duplicate email detection is handled gracefully with a 409 conflict response.
- Atomic User Creation: If Firebase Auth creation succeeds but Firestore write fails, the Auth user is cleaned up to prevent orphaned accounts.

5.2.2 Payroll Calculation Algorithm

The payroll calculation module computes the monthly salary for all active employees. The algorithm fetches active employee records, applies the base salary, and stores the result in Firestore for that pay period.

Pseudocode:

```
START   SELECT month, year from input   FETCH all employees WHERE status =
"active"      FOR each employee:     basic_salary  <- employee.salary
gross_salary  <- basic_salary + allowances     deductions    <- PF + ESI +
TDS     net_salary    <- gross_salary - deductions          STORE
payroll_record(employee_id, month, year,
gross_salary, deductions, net_salary)   END FOR      RETURN success END
```

5.2.3 Code Efficiency and Optimization Techniques

Code efficiency was a critical consideration to ensure the system performs well under concurrent usage. The following optimization strategies were applied:

- Firebase Firestore Indexing: Composite indexes were created on employee status and department fields to enable fast filtered queries.
- Server-Side Rendering (SSR): Next.js SSR was used for dashboard and reporting pages to minimize client-side computation.
- API Route Validation: All API routes validate input before any database operation, reducing unnecessary Firestore read/write costs.
- Role-Based Access Control: Firebase security rules enforce that only admins can write to employee and payroll collections, preventing unauthorized mutations.
- Reusable Components: React components for employee cards, payslip views, and leave tables are modularized for reuse across pages.
- Environment Variables: Sensitive configuration (Firebase credentials) is stored in .env files and never exposed to the client bundle.

5.3 Testing Approach

Testing was conducted following a multi-level testing strategy in alignment with the Agile methodology. Both functional and non-functional testing techniques were applied to ensure correctness, reliability, and performance. The testing approach adopted for this project includes Unit Testing, Integration Testing, Alpha Testing, and Beta (User Acceptance) Testing.

5.3.1 Unit Testing

Unit testing is a software testing technique that tests individual software units such as individual functions, components, or API routes in isolation to verify their correctness. Each module was tested independently by the developer during the sprint to detect defects early.

Modules covered in unit testing:

- User Authentication Module — Login, logout, and session management
- Employee Management Module — Create, read, update, and delete operations
- Payroll Generation Module — Salary computation logic
- Leave Request Module — Leave submission and approval workflow
- Attendance Module — Daily attendance marking and retrieval

Example Unit Test Scenarios:

- Input validation for missing required fields during employee creation.
- Password minimum length enforcement at the API layer.
- Duplicate email detection and correct HTTP 409 response.
- Leave status transitions (Pending to Approved or Rejected).

5.3.2 Integration Testing

Integration testing focuses on verifying the correctness of interfaces and data flow between connected modules. Once all modules were unit-tested, integration testing was performed to ensure that the complete system works end-to-end without errors.

Integration test scenarios included:

- Admin login — Employee creation — Payroll generation flow.
- Employee login — Leave request submission — Admin approval.
- Attendance marking — Payroll deduction computation.
- Firebase Auth token validation across all protected API routes.

The system was tested for data consistency across Firestore collections, API interoperability between frontend and backend, and correct error propagation from the data layer to the UI.

5.3.3 Alpha Testing

Alpha Testing is performed internally by the development team and selected quality assurance testers to identify bugs before releasing the product to real users. Alpha testing was performed near the end of the development phase to validate functional correctness, UI consistency, error handling, and system stability under controlled conditions.

Done by : Manthan Taak

5.3.4 Beta Testing (User Acceptance Testing)

Beta Testing was performed by real users in a real environment to obtain feedback on product quality. A beta version of the Payroll Management System was released to a limited number of end-users to validate usability, performance, and overall experience.

Feedback areas evaluated:

- UI responsiveness across devices and screen sizes.
- Clarity of error messages for incorrect login credentials.
- Ease of payroll generation and leave approval workflows.
- Performance under multiple concurrent requests.

Feedback obtained during beta testing was incorporated into subsequent improvements including enhanced error messages, UI layout refinements, and additional form validations.

Done By : Vikesh Chiluka & Prerak Sidhpura

5.4 Modifications and Improvements

Based on the results of testing, multiple refinements were implemented to improve system robustness, usability, and correctness. The following table summarizes the key issues identified and the modifications implemented:

Identified Issue	Modification Implemented	Improvement Achieved
Duplicate employee email allowed	Added email uniqueness check at API layer	Data integrity enforced
No feedback on login failure	Added error toast with Firebase auth error code	Better user experience
Payroll generated multiple times	Added idempotency check on month/year per employee	Prevented duplicate records
Leave approval did not update status	Fixed Firestore document update logic	Correct status transitions
Admin could delete own account	Added self-delete prevention check	System stability improved
Slow dashboard load	Firestore query optimized with indexes	Reduced load time

5.5 Test Cases

The following test cases were defined and executed to validate the system across key functional areas:

5.5.1 Authentication Test Cases

Test Case ID	Description	Input	Expected Output	Status
TC-AUTH-01	Valid admin login	Correct email & password	Redirect to HR Dashboard	PASS
TC-AUTH-02	Invalid credentials	Wrong password	Error: auth/invalid-credential	PASS
TC-AUTH-03	Empty fields login	Blank email & password	Validation error shown	PASS
TC-AUTH-04	Session persistence	Refresh page after login	User remains logged in	PASS

Test Case ID	Description	Input	Expected Output	Status
TC-AUTH-05	Logout	Click logout button	Session cleared, redirect to login	PASS

5.5.2 Employee Management Test Cases

Test Case ID	Description	Input	Expected Output	Status
TC-EMP-01	Add new employee	All valid fields	Employee created in Firestore	PASS
TC-EMP-02	Duplicate email	Existing email address	HTTP 409 Conflict error	PASS
TC-EMP-03	Missing required field	Empty salary field	Validation error message	PASS
TC-EMP-04	View employee list	Admin navigates to Employees	All employees displayed	PASS
TC-EMP-05	Delete employee	Click delete on employee card	Employee removed from list	PASS

5.5.3 Payroll Test Cases

Test Case ID	Description	Input	Expected Output	Status
TC-PAY-01	Generate payroll	Select month: March 2026	Payroll created for all active employees	PASS
TC-PAY-02	Duplicate payroll generation	Same month re-generated	Existing record updated, no duplicate	PASS
TC-PAY-03	View payslip	Employee views their payslip	Correct salary breakdown displayed	PASS
TC-PAY-04	Payroll for inactive employee	Inactive employee in system	Employee excluded from payroll	PASS

5.5.4 Leave Management Test Cases

Test Case ID	Description	Input	Expected Output	Status
TC-LVE-01	Submit leave request	Valid date range and reason	Leave saved as Pending	PASS
TC-LVE-02	Approve leave request	Admin clicks approve	Status updated to Approved	PASS
TC-LVE-03	Reject leave request	Admin clicks reject	Status updated to Rejected	PASS
TC-LVE-04	View pending requests	Admin views leave table	All pending leaves visible	PASS

Chapter 6: Results and Discussion

6.1 Test Reports

The test results derived from the defined test cases demonstrate that the Payroll Management System is highly capable of handling diverse operational scenarios and functions reliably under various conditions. Both positive and negative test scenarios were executed to validate that the system correctly handles valid inputs, edge cases, and erroneous inputs.

6.1.1 Authentication Test Report

This phase of testing ensures that user access is secure and that the system correctly handles both valid and invalid login attempts. Firebase Authentication is used to manage credentials, ensuring password hashing and token-based session management.

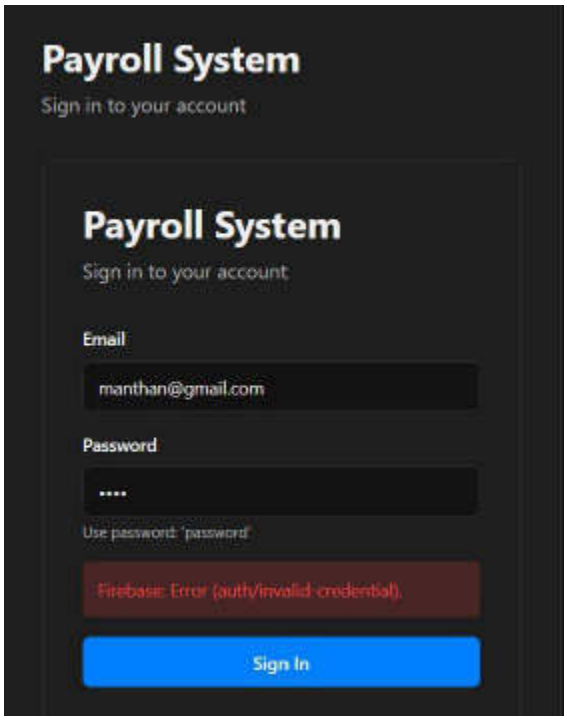

Figure 6.1 — Login failure with invalid credentials (Firebase: auth/invalid-credential)

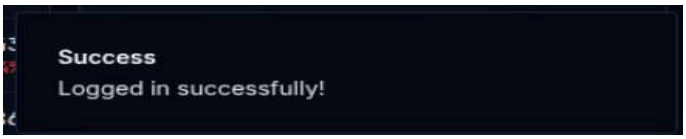

Figure 6.2 — Successful login confirmation toast

Test ID	Objective	Input	Expected Result	Observed Result	Status
AUTH-01	Valid admin login	Correct email & password	Redirect to Dashboard	Successfully routed to HR Dashboard	PASS
AUTH-02	Invalid credentials	Wrong password entered	Show auth error	Firebase error displayed	PASS
AUTH-03	Empty form submission	Blank fields	Validation warning	Required field warnings appeared	PASS
AUTH-04	Session persistence	Page refresh after login	Stay logged in	User remained authenticated	PASS
AUTH-05	Logout functionality	Click logout	Session cleared	Redirected to login page	PASS

6.1.2 HR Dashboard Test Report

The HR Dashboard is the primary landing page for administrators after login. It displays key metrics including total employees, monthly payroll, pending requests, and today's attendance. Testing verified the correctness of all displayed values.

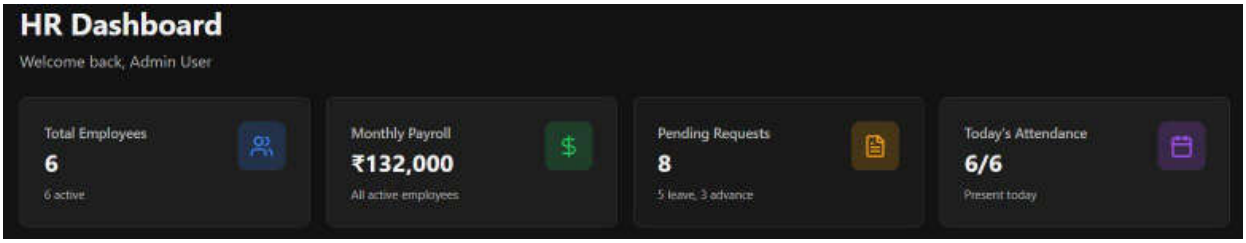

Figure 6.3 — HR Dashboard showing key metrics

Test ID	Objective	Expected Result	Observed Result	Status
UI-01	Total employee count accuracy	Count matches Firestore employees	6 active employees displayed correctly	PASS

Test ID	Objective	Expected Result	Observed Result	Status
UI-02	Monthly payroll sum	Sum of all active employee salaries	Rs. 1,32,000 displayed accurately	PASS
UI-03	Pending requests count	Count of pending leave + advances	8 pending (5 leave, 3 advance) shown	PASS
UI-04	Attendance display	Today's present count vs total	6/6 present today shown correctly	PASS
UI-05	Dashboard data on login	Data loads on page mount	Metrics populated immediately on login	PASS

6.1.3 Payroll Generation Test Report

The payroll generation module is the core functional component of the system. Administrators select a month and year, and the system generates payroll records for all active employees. Testing covered positive generation, duplicate prevention, and UI feedback.

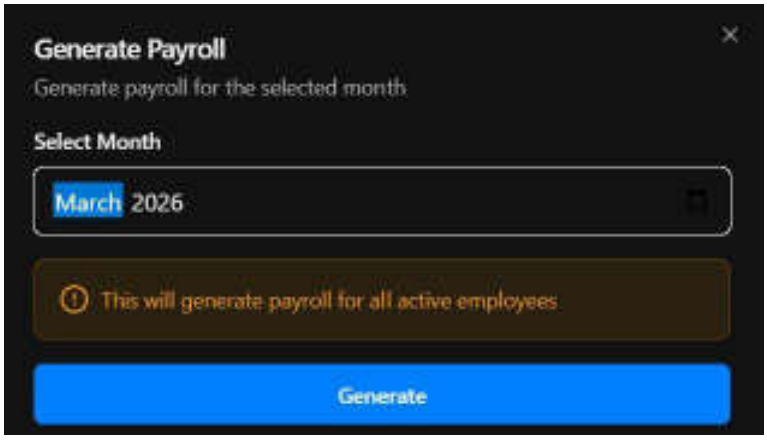

Figure 6.4 — Generate Payroll dialog for March 2026

Test ID	Objective	Input	Expected Result	Observed Result	Status
PAY-01	Generate payroll for active employees	March 2026 selected	Payroll records created for all active employees	Payroll generated for 6 employees	PASS
PAY-02	Prevent duplicate generation	Same month re-submitted	No duplicate Firestore record	Existing record updated, no duplicate	PASS
PAY-03	Warning message shown	Dialog opened	Info banner visible before generating	Orange warning banner displayed	PASS

Test ID	Objective	Input	Expected Result	Observed Result	Status
PAY-04	Inactive employees excluded	System has inactive employee	Only active employees in payroll	Inactive employee correctly excluded	PASS

6.1.4 Leave Management Test Report

The leave management module allows employees to submit leave requests and administrators to approve or reject them. Testing validated the complete workflow from submission to status update.

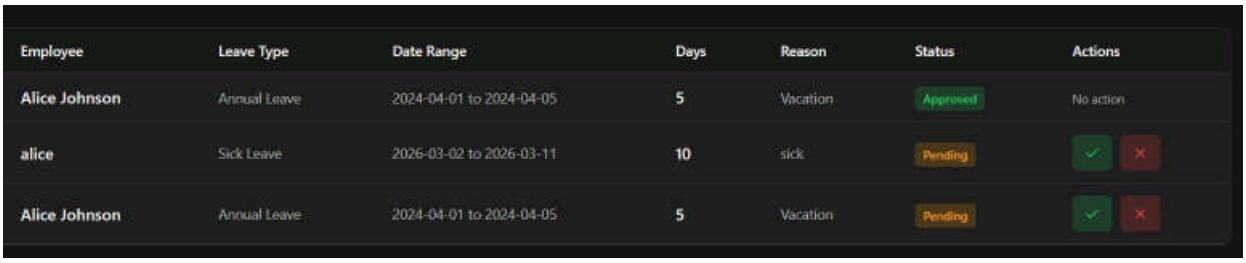

Figure 6.5 — Leave management table with Approved and Pending statuses

Test ID	Objective	Input	Expected Result	Observed Result	Status
LVE-01	Submit leave request	Date range: 2026-03-02 to 2026-03-11, reason: sick	Leave saved with Pending status	Leave correctly saved as Pending	PASS
LVE-02	Approve leave	Admin clicks approve (tick button)	Status changed to Approved	Status updated, no action buttons shown	PASS
LVE-03	Reject leave	Admin clicks reject (cross button)	Status changed to Rejected	Leave removed from pending list	PASS
LVE-04	View leave history	Admin navigates to leave module	All records with status badges	All records displayed correctly	PASS

6.2 User Documentation

This section provides a walkthrough of the Payroll Management System's key modules and screens. The documentation is intended to guide any user — administrator or employee — in understanding the system's functionality and operation.

6.2.1 Login Screen

The login screen is the entry point for all users. Users enter their registered email address and password to access the system. On invalid credentials, the system displays a Firebase authentication error message. On successful login, a green success toast notification is shown and the user is redirected to their dashboard.

Steps to login:

- Navigate to the Payroll System URL.
- Enter your registered email address in the Email field.
- Enter your password in the Password field.
- Click the Sign In button.
- On success, you will be redirected to the HR Dashboard.

6.2.2 HR Dashboard

After a successful login, the administrator is presented with the HR Dashboard. The dashboard provides a high-level summary of the organization's payroll status, including total active employees, total monthly payroll value, the number of pending requests (leave and advance), and today's attendance count. This screen serves as the control center for all payroll operations.

6.2.3 Payroll Generation

The Payroll Generation module allows HR administrators to generate monthly payroll for all active employees in a single operation. The administrator selects the desired month and year from the date picker, then clicks Generate. The system computes and stores payroll records for all active employees for that pay period. A confirmation warning is shown before the action is executed to prevent accidental generation.

6.2.4 Leave Management

The Leave Management module displays all employee leave requests in a tabular format. Each record shows the employee name, leave type, date range, number of days, reason, current status (Pending/Approved/Rejected), and action buttons. Administrators can approve or reject pending requests using the tick (approve) and cross (reject) action buttons. Once a decision is made, the action buttons are hidden and the status badge is updated accordingly.

6.2.5 Employee Management

The Employee Management module allows administrators to add new employees, view existing employee records, and manage employee profiles. When creating a new employee, the system creates both a Firebase Authentication account and a Firestore employee document simultaneously. The module also supports editing employee details and deactivating employees when they leave the organization.

Chapter 7: Conclusions

7.1 Conclusion

The Payroll Management System was successfully designed, developed, and tested as a full-stack web application using Next.js, Firebase, and Tailwind CSS. The system addresses the core challenges of manual payroll processing by automating salary computation, attendance tracking, leave management, and report generation within a secure and user-friendly interface.

Through the implementation of role-based access control, the system ensures that only authorized administrators can manage employee records and generate payroll, while employees can securely view their payslips and submit leave requests. The use of Firebase Authentication and Firestore guarantees data security, real-time synchronization, and reliable storage.

The Agile development methodology facilitated iterative development and continuous testing, ensuring that each module was validated before integration. Comprehensive unit, integration, alpha, and beta testing demonstrated that the system handles both standard workflows and edge cases — such as duplicate emails, invalid credentials, and concurrent payroll generation — gracefully and correctly.

Overall, the project successfully fulfills all stated objectives: automating payroll processing, minimizing human errors, providing secure access to employee data, and generating reliable financial reports. The system is a practical, scalable solution applicable to small and medium-sized organizations seeking to modernize their HR and payroll operations.

7.2 Limitations of the System

While the Payroll Management System achieves its core objectives, the following limitations were identified during development and testing:

- Internet Dependency: The system requires a stable internet connection since all data is stored in Firebase Firestore. Offline operation is not supported in the current implementation.

- Scalability for Large Enterprises: The current architecture using Firebase's free-tier Firestore may encounter rate limits and performance degradation for organizations with more than 500 employees and high-frequency payroll operations.
- No Advanced Tax Engine: The current version supports basic salary and deduction computation. Complex statutory compliance modules (TDS slabs, ESI brackets, professional tax) are not fully automated.
- PDF Payslip Generation: Downloadable PDF payslips are partially implemented. The current version displays payslip data on-screen but does not support client-side PDF export in all browsers.
- Single Tenant Architecture: The system currently supports a single organization. Multi-tenant support for payroll service providers managing multiple organizations is not yet implemented.
- No Audit Trail: Detailed audit logs tracking every payroll modification, approval action, and employee record change are not yet implemented.

7.3 Future Scope of the Project

The following enhancements are identified as potential areas for future development:

- Advanced Tax Compliance Engine: Integration with government tax APIs to automate TDS computation, Form 16 generation, and GST compliance for contract employees.
- AI-Powered Attendance Tracking: Integration of facial recognition or biometric attendance systems with automatic payroll deduction for absences.
- Predictive Salary Forecasting: Use of machine learning models to forecast payroll costs based on historical trends, seasonality, and business growth projections.
- Multi-Tenant Architecture: Refactoring the system to support multiple organizations under a single deployment, enabling payroll service providers to onboard multiple clients.
- Mobile Application: Development of a React Native mobile application allowing employees to view payslips, mark attendance, and submit leave requests from mobile devices.

- PDF and Excel Export: Implementation of server-side PDF generation and Excel export for payroll registers and financial reports.
- Audit Trail and Activity Logs: A comprehensive logging module tracking every system action with timestamps and user attribution.
- Integration with Accounting Systems: API integration with accounting platforms such as Tally, Zoho Books, or QuickBooks for automated journal entry posting on payroll processing.

References

1. Facebook Inc. (2023). React — A JavaScript library for building user interfaces. https://react.dev/

2. Vercel Inc. (2024). Next.js Documentation. https://nextjs.org/docs

3. Google LLC. (2024). Firebase Documentation — Authentication and Firestore. https://firebase.google.com/docs

4. Tailwind Labs. (2024). Tailwind CSS Documentation. https://tailwindcss.com/docs

5. Tiobe Software. (2024). TypeScript Programming Language Guide. https://www.typescriptlang.org/docs/

6. Beck, K., et al. (2001). Manifesto for Agile Software Development. https://agilemanifesto.org/

7. Pressman, R. S., & Maxim, B. R. (2015). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.

8. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.

9. MDN Web Docs. (2024). HTTP — HyperText Transfer Protocol Reference. https://developer.mozilla.org/en-US/docs/Web/HTTP

10. Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.

11. Google LLC. (2024). Firebase Security Rules Documentation. https://firebase.google.com/docs/rules

12. Node.js Foundation. (2024). Node.js Official Documentation. https://nodejs.org/en/docs

Glossary

Term / Acronym	Definition
API	Application Programming Interface — a set of defined rules enabling communication between software applications.
Firebase	A Backend-as-a-Service (BaaS) platform by Google providing authentication, NoSQL database, and hosting services.
Firestore	Google Firebase's scalable NoSQL cloud database that stores and syncs data in real time.
Next.js	A React-based full-stack web framework by Vercel supporting server-side rendering and API routes.
React	A JavaScript library by Facebook for building component-based user interfaces.
SSR	Server-Side Rendering — generating HTML on the server before sending it to the client.
JWT	JSON Web Token — a compact, URL-safe token used for stateless authentication.
HRMS	Human Resource Management System — software for managing employee information and HR operations.
PF	Provident Fund — a mandatory employee retirement savings scheme in India.
TDS	Tax Deducted at Source — income tax deducted at the point of salary payment.
ESI	Employee State Insurance — a social security scheme for Indian employees.
SDLC	Software Development Life Cycle — the structured process for planning, creating, testing, and deploying software.
UAT	User Acceptance Testing — testing performed by end-users to verify the system meets requirements.
CRUD	Create, Read, Update, Delete — the four basic operations performed on database records.
NoSQL	Not Only SQL — a non-relational database model, as used by Firebase Firestore.